



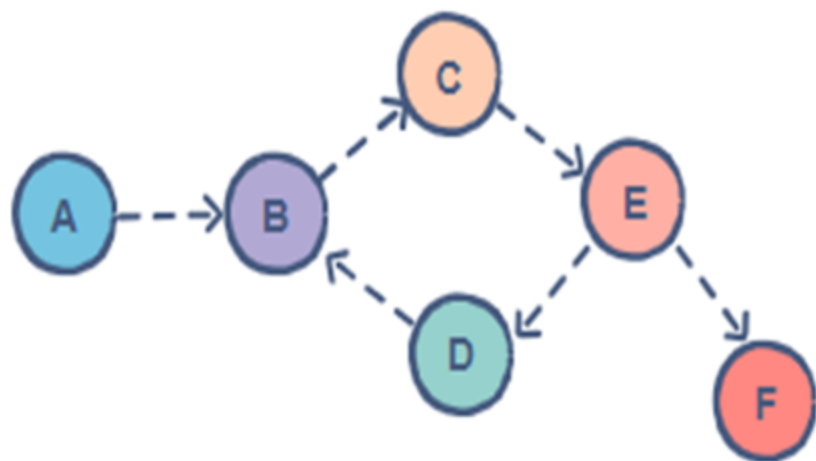
Graph Data Structure

What will be covered

- Concept Of Graph
- Need and Application of Graph
- Types of Graph
- Graph Representation.
- Adjacency Matrix.

- A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.
- A graph data structure is a collection of nodes that have data and are connected to other nodes.
- Let's try to understand this through an example. On facebook, everything is a node. That includes User, Photo, Album, Event, Group, Page, Comment, Story, Video, Link, Note...anything that has data is a node.
- Every relationship is an edge from one node to another. Whether you post a photo, join a group, like a page, etc., a new edge is created for that relationship.

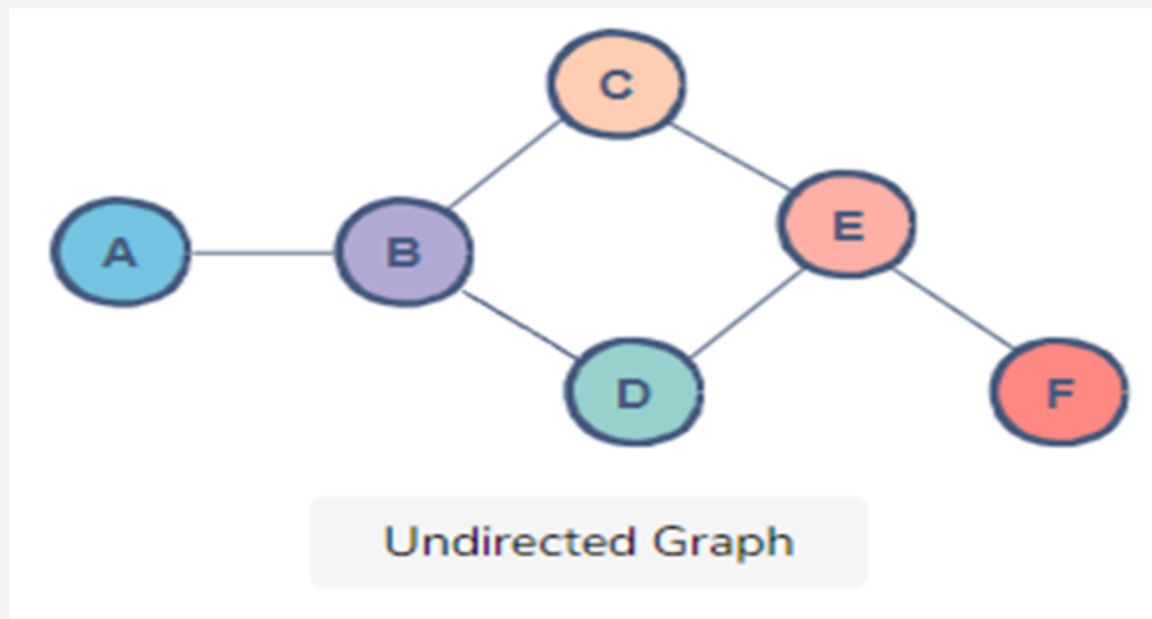
- Facebook: Each user is represented as a vertex and two people are friends when there is an edge between two vertices. Similarly friend suggestion also uses graph theory concept.
- Google Maps: Various locations are represented as vertices and the roads are represented as edges and graph theory is used to find shortest path between two nodes.
- Recommendations on e-commerce websites: The “Recommendations for you” section on various e-commerce websites uses graph theory to recommend items of similar type to user’s choice.



Directed Graph

A Directed graph is a set of vertices (nodes) connected by edges, with each node having a direction associated with it.

Edges are usually represented by arrows pointing in the direction the graph can be traversed.



In an undirected graph the edges are bidirectional, with no direction associated with them. Hence, the graph can be traversed in either direction. The absence of an arrow tells us that the graph is undirected.

We can represent a graph in many ways. The two most common ways of representing a graph is as follows:

Adjacency matrix

An adjacency matrix is a $V \times V$ binary matrix A . Element A_{ij} is 1 if there is an edge from vertex i to vertex j else A_{ij} is 0.

Note: A binary matrix is a matrix in which the cells can have only one of two possible values - either a 0 or 1.

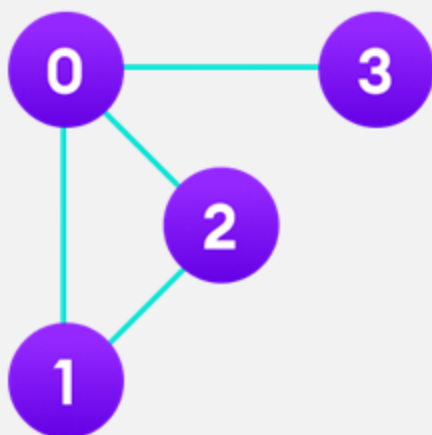
Adjacency list

The other way to represent a graph is by using an adjacency list. An adjacency list is an array A of separate lists. Each element of the array A_i is a list, which contains all the vertices that are adjacent to vertex i .

Adjacency Matrix

In adjacency matrix is a 2D array of $V \times V$ vertices. Each row and column represent a vertex.

If the value of any element $a[i][j]$ is 1, it represents that there is an edge connecting vertex i and vertex j .



	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	0
3	1	0	0	0

We use the following steps to implement BFS traversal...

- Step 1 - Define a Queue of size total number of vertices in the graph.
- Step 2 - Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.
- Step 3 - Visit all the non-visited adjacent vertices of the vertex which is at front of the Queue and insert them into the Queue.
- Step 4 - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.
- Step 5 - Repeat steps 3 and 4 until queue becomes empty.
- Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

We use the following steps to implement DFS traversal...

- Step 1- Start by putting any one of the graph's vertices on top of a stack.
- Step 2-Take the top item of the stack and add it to the visited list.
- Step 3-Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
- Step 4-Keep repeating steps 2 and 3 until the stack is empty.



Graph Data Structure

- Adjacency List
- Advantages and Disadvantages .
- Implementation of DFS Traversal using Stack .
- Implementation of BFS Traversal using Queue.

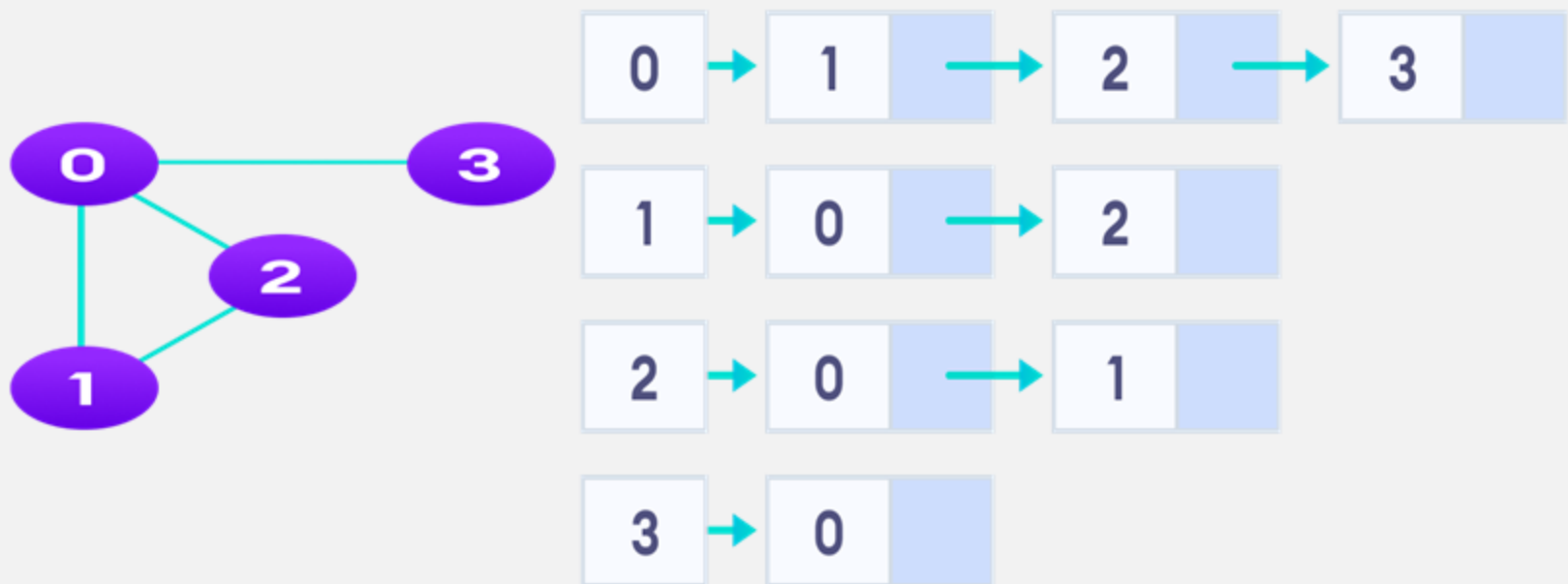
Advantages Of Adjacency List

- An adjacency list is efficient in terms of storage because we only need to store the values for the edges. For a sparse graph with millions of vertices and edges, this can mean a lot of saved space.
- It also helps to find all the vertices adjacent to a vertex easily.

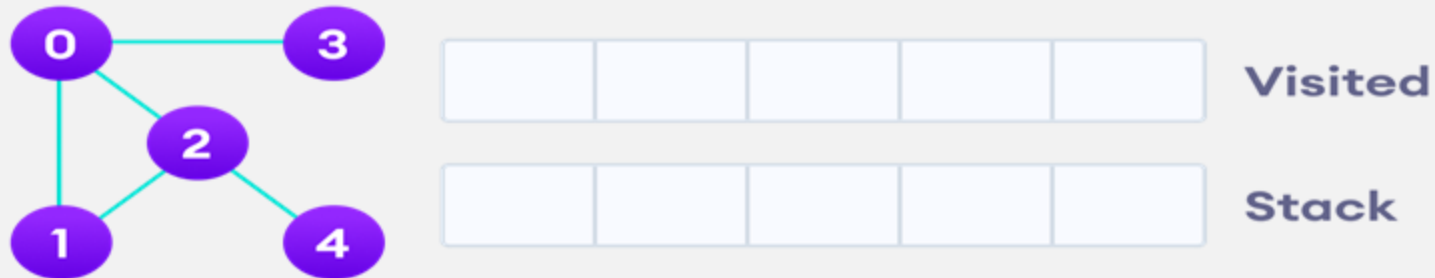
Disadvantages of Adjacency List

- Finding the adjacent list is not quicker than the adjacency matrix because all the connected nodes must be first explored to find them.

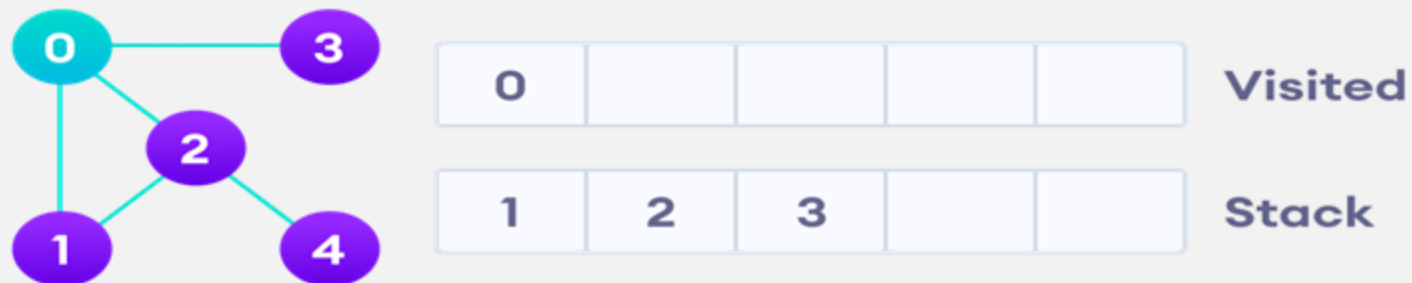
- An adjacency list represents a graph as an array of linked lists. The index of the array represents a vertex and each element in its linked list represents the other vertices that form an edge with the vertex.



DFS Traversal Using Stack

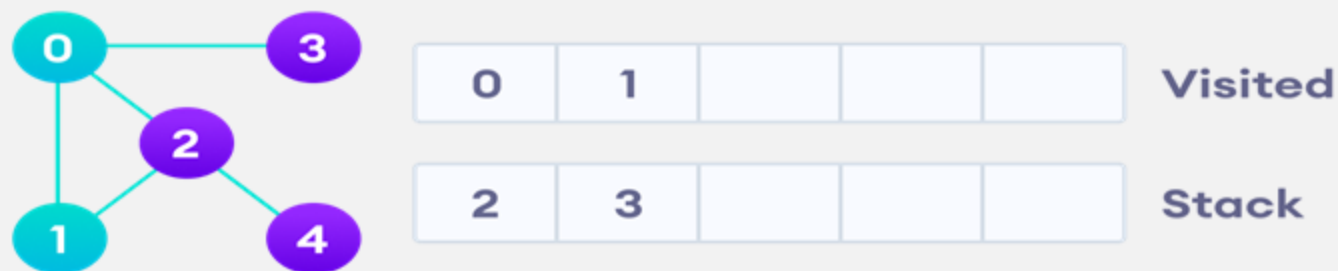


We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.

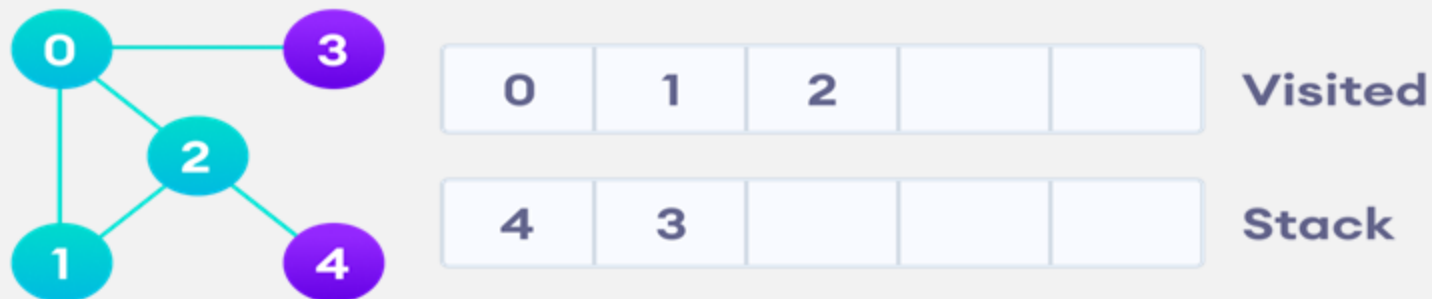


DFS Traversal Using Stack

Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

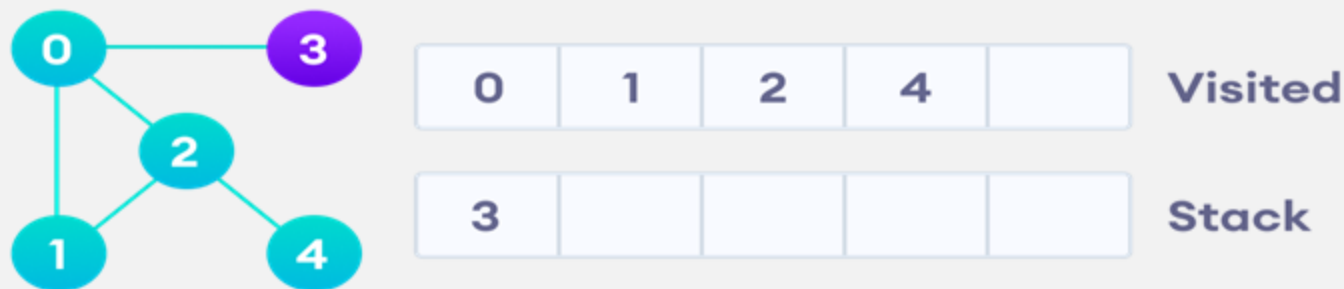


Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

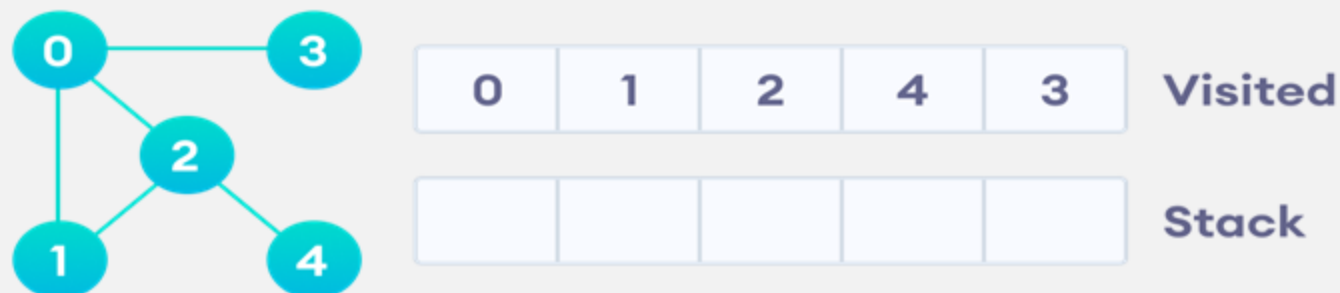


DFS Traversal Using Stack

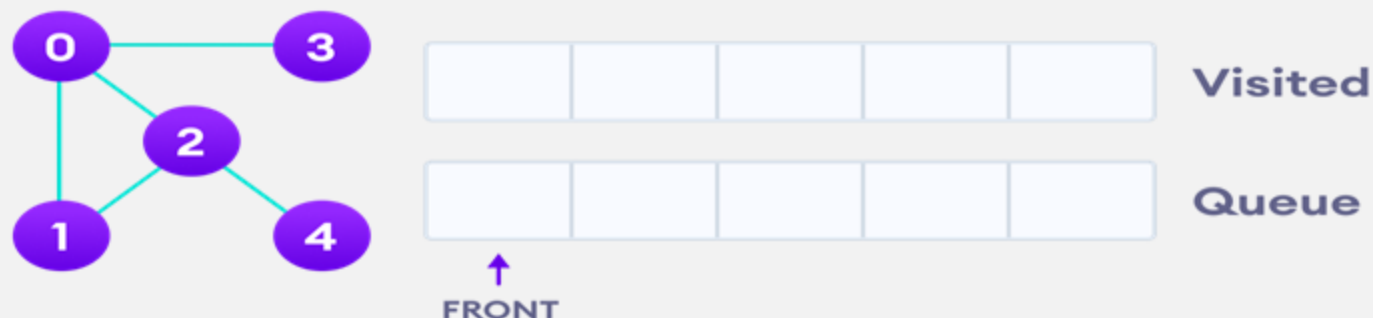
Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.



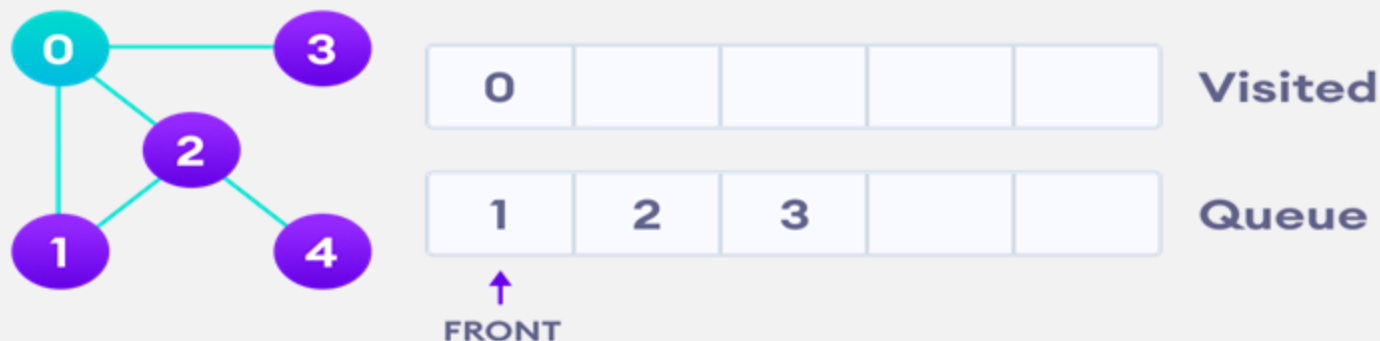
After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.



BFS Traversal Using Queue

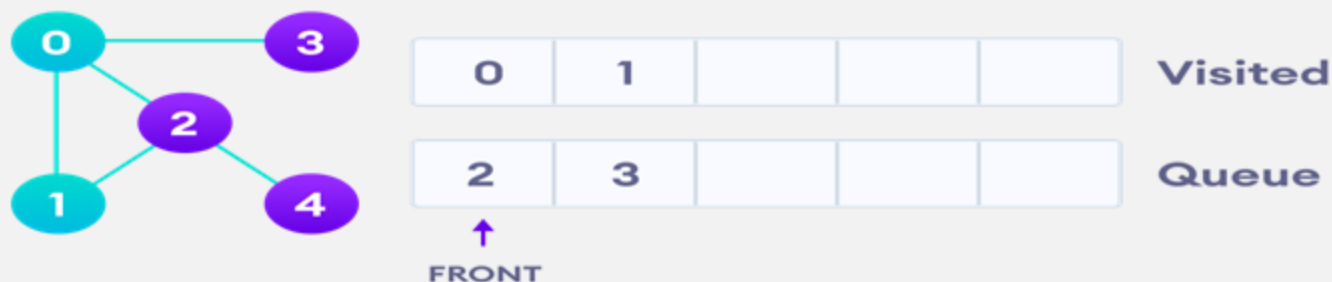


We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.

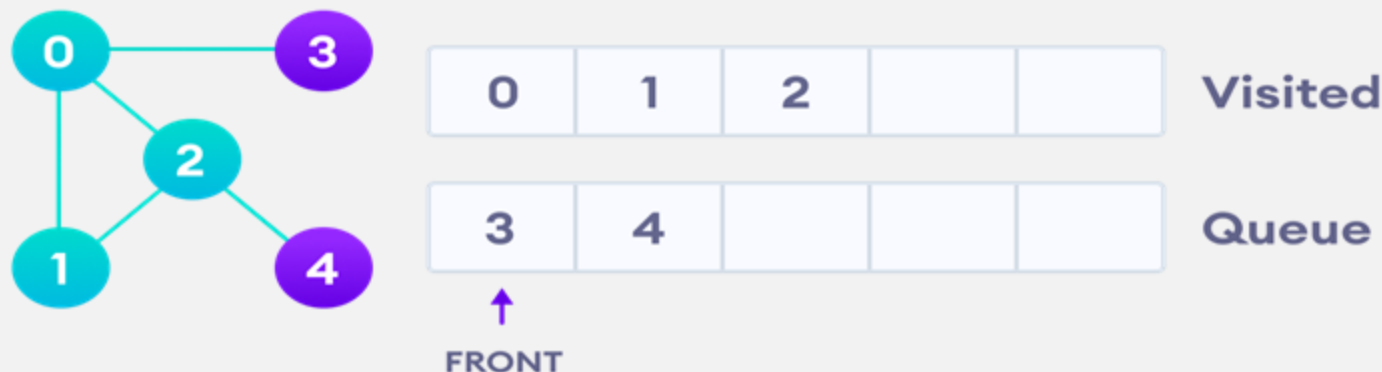


BFS Traversal Using Queue

Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

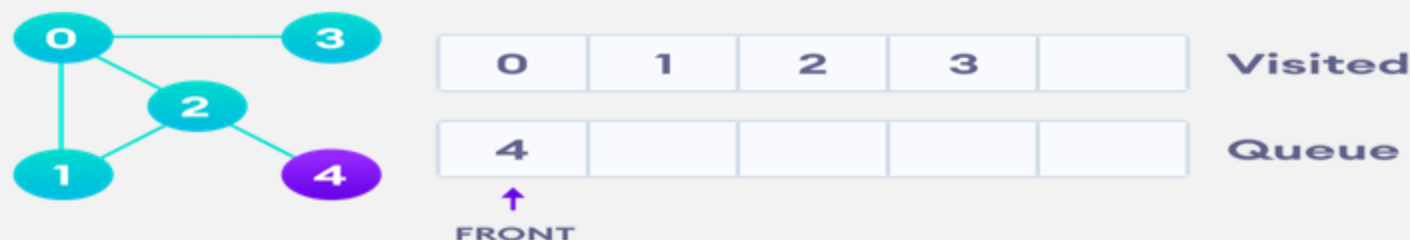


Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.



BFS Traversal Using Queue

Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.

